8051 Microcontroller: Lecture 6 Notes

Arithmetic (ADD, ADDC, SUBB, MUL, DIV) and logical (ANL, ORL, XRL) instructions.

Conventions: A = accumulator, B = B register, CY = carry flag, AC = auxiliary carry flag, OV = overflow flag, H = hexadecimal.

The ADDC Instruction

8051 Microcontroller: Add with Carry

1. What ADDC does

ADDC means "add with carry". It adds the source operand and the carry flag (CY) to the accumulator, and stores the result in the accumulator:

```
ADDC  A, source        ; A = A + source + CY
```

Source: a register (R0 to R7), a direct address, an indirect address (@R0 or @R1), or an immediate number (#data).

Flags affected: CY, AC, OV, and P.

Flag	Meaning after ADDC
CY	1 if there is a carry out of bit 7 (the result is larger than FFH), otherwise 0.
AC	1 if there is a carry from bit 3 to bit 4, otherwise 0.
OV	1 if there is a signed overflow, otherwise 0.
P	1 if the number of 1s in A is odd, otherwise 0 (parity).

2. ADD versus ADDC

Instruction	Operation	Uses old CY?
ADD A, source	A = A + source	No
ADDC A, source	A = A + source + CY	Yes

> **Rule:** When adding multi-byte numbers, use ADD for the lowest byte, then ADDC for each higher byte.

3. Example 1: the effect of the carry flag

Both programs add 5 and 3. The only difference is the value of CY before ADDC.

Example 1a: carry cleared (CLR C)

```
CLR   C
MOV   A, #5
ADDC  A, #3       ; A = 05H + 03H + 0 = 08H
```

```
    0000 0101      (05H)
  + 0000 0011      (03H)
  +           0    (CY)
  -----------
    0000 1000      (08H)
```

Result: A = 08H, CY = 0, AC = 0, OV = 0, P = 1 (08H has one 1).

Example 1b: carry set (SETB C)

```
SETB  C
MOV   A, #5
ADDC  A, #3        ; A = 05H + 03H + 1 = 09H

    0000 0101      (05H)
  + 0000 0011      (03H)
  +           1    (CY)
  -----------
    0000 1001      (09H)
```

Result: A = 09H, CY = 0, AC = 0, OV = 0, P = 0 (09H has two 1s).

> **Lesson:** the program is identical except for CLR C or SETB C, yet the answer changes from 08H to 09H.

4. Example 2: add 02C4H and 0159H

Task: write a program that adds the 16-bit numbers 02C4H and 0159H. Save the result in R6 (low byte) and R7 (high byte).

Step 1: split each number into bytes

Number	High byte	Low byte
02C4H	02H	C4H
0159H	01H	59H

Step 2: add the low bytes first then add the high bytes plus the carry

```
      1      (CY from the low bytes)
    02 C4H
  + 01 59H
    ---------
    04 1DH
```

Add one hex digit at a time, starting from the right: 4 + 9 = D. Then C + 5 = 12 + 5 = 17 decimal = 11H, so write 1 and carry 1. The sum is 1DH. The low byte of the result is 1DH, and the extra 1 is a carry out of the byte, so CY = 1. This carry must be added into the high bytes.

02H + 01H + 1 = 04H. There is no further carry, so CY = 0.

Program

```
CLR   C             ; start with no carry
MOV   A, #0C4H      ; A = low byte of 02C4H
ADDC  A, #59H       ; A = 1DH, CY = 1
MOV   R6, A         ; R6 = 1DH (low byte of result)
MOV   A, #02H       ; A = high byte of 02C4H
ADDC  A, #01H       ; A = 02H + 01H + CY = 04H
MOV   R7, A         ; R7 = 04H (high byte of result)

SJMP $
```

Result: R7:R6 = 041DH, so R6 = 1DH and R7 = 04H.

Check: 02C4H = 708 and 0159H = 345 in decimal. 708 + 345 = 1053, and 1053 = 041DH.

2. Subtraction: SUBB

The 8051 has no plain subtract instruction. It only has SUBB, which means subtract with borrow:

```
SUBB  A, source         ; A = A - source - CY
```

Flags affected: CY, AC, P and OV.

Rule: because SUBB also subtracts the carry flag, you must clear the carry (CLR C) before a normal subtraction.

Example 1: Write a code to subtract 5 from 9 (9 – 5)

```
MOV   A, #9
CLR   C
SUBB  A, #5        ; A = 09H - 05H - 0 = 04H
```

Example 2: 9 - 5 with CY = 1

```
MOV   A, #9
SETB  C
SUBB  A, #5        ; A = 09H - 05H - 1 = 03H
```

> Example 2 shows what happens if you forget to clear the carry: the answer is off by one. This is a common source of bugs.

Steps the CPU performs to do SUBB

1. Take the 2's complement of the source.
2. Add it to A.
3. Invert CY and AC.

Reading the result:

Final CY	Meaning
If CY = 0	The result is positive (or zero). A holds the answer directly.
If CY = 1	The result is negative. A holds the answer in 2's complement form.

Worked example: show CY, AC and A after executing the following code

```
MOV   A, #5
CLR   C
SUBB  A, #9
```

Step 1. 2's complement of the source. `09H = 0000 1001` becomes `1111 0111`.

Step 2. Add it to A.

```
    0000 0101      (A = 05H)
  + 1111 0111      (2's complement of 09H)
  -----------
    1111 1100      AC = 0, CY = 0 (no carries)
```

Step 3. Invert CY and AC. CY becomes 1 and AC becomes 1.

CY	AC	A
1	1	1111 1100 = FCH = -4 (the result is negative)

> To read a negative result, take the 2's complement again: FCH = - 4 (signed), so A holds - 4. CY = 1 and AC = 1

4. Packed and unpacked BCD

BCD (binary coded decimal) stores a decimal number one digit at a time. Each decimal digit 0 to 9 uses 4 bits (0000 to 1001). The codes 1010 to 1111 (AH to FH) are not valid BCD digits.

Type	Storage	Example: decimal 59
Packed BCD	Two digits in one byte. Range: 00 to 99.	One byte: 0101 1001 = 59H
Unpacked BCD	**One digit** in **one byte**. The upper nibble is always 0000.	Two bytes: 05H and 09H (0000 0101 and 0000 1001)

Packed BCD saves memory and is used for arithmetic with DA A (next section). Unpacked BCD is easier to convert to ASCII: add 30H to each digit (05H becomes 35H, the ASCII code of "5"), as in the logical instructions notes.

6. The DA A instruction (decimal adjust)

The 8051 adds in binary. If you add two packed BCD numbers with ADD or ADDC, the result in A can be a wrong BCD number. For example, 27H + 15H = 3CH, and C is not a BCD digit. DA A (decimal adjust accumulator) corrects the result so that A holds packed BCD again.

```
DA   A              ; adjust A after ADD or ADDC
```

Rules. DA A checks the result in two steps:

Step	Condition	Action
1	Low nibble of A > 9, or AC = 1	Add 06H to A
2	High nibble of A > 9, or CY = 1	Add 60H to A, and set CY = 1

In other words: The DA A instruction adds 6 to the lower nibble if the lower nibble > 9

or AC = 1, and adds 6 to the upper nibble if the upper nibble > 9 or CY = 1.

- **Use it only right after ADD or ADDC** and only for packed BCD. DA A does not work correctly after SUBB, INC, or DEC.
- **CY after DA A:** CY = 1 means the decimal sum is 100 or more. DA A can set CY but never clears it.

7. Examples covering every DA A case

Each example adds two packed BCD numbers with ADD and then adjusts the result with DA A. The decimal sum is shown in the first line of each result.

Case 1: no adjustment needed (25 + 34 = 59)

```
MOV   A, #25H
ADD   A, #34H      ; A = 59H, AC = 0, CY = 0
DA    A            ; A = 59H (unchanged)
```

Both nibbles are 9 or less, AC = 0, and CY = 0, so DA A changes nothing.

Case 2: low nibble greater than 9 (27 + 15 = 42)

```
MOV   A, #27H
ADD   A, #15H      ; A = 3CH, AC = 0, CY = 0
DA    A            ; C > 9, so add 06H: A = 42H
```

Result: A = 42H, CY = 0.

Case 3: AC = 1 with a valid low nibble (29 + 18 = 47)

```
MOV   A, #29H
ADD   A, #18H      ; A = 41H, AC = 1 (9 + 8 = 11H)
DA    A            ; AC = 1, so add 06H: A = 47H
```

The low nibble 1 looks valid, but the carry from bit 3 (AC = 1), so DA A still adds 06H. Result: A = 47H, CY = 0.

Case 4: high nibble greater than 9 (52 + 73 = 125)

```
MOV   A, #52H
ADD   A, #73H      ; A = C5H, AC = 0, CY = 0
DA    A            ; C > 9, so add 60H: A = 25H, CY = 1
```

Result: CY=1 A = 25H, which is 1 25H.

Case 5: CY = 1 with a valid high nibble (91 + 92 = 183)

```
MOV   A, #91H
ADD   A, #92H      ; A = 23H, CY = 1
DA    A            ; CY = 1, so add 60H: A = 83H, CY = 1
```

The binary sum overflowed the byte, so CY = 1 and the high nibble is corrected even though 2 is a valid digit. Result: CY=1 A = 83H, which is 1 83H.

Case 6: both adjustments (78 + 69 = 147)

```
MOV   A, #78H
ADD   A, #69H      ; A = E1H, AC = 1, CY = 0
DA    A            ; AC = 1, add 06H: E7H; E > 9, add 60H
                   ; A = 47H, CY = 1
```

Result: is 1 47H.

Case 7: AC = 1 and CY = 1 together (99 + 99 = 198)

```
MOV   A, #99H
ADD   A, #99H      ; A = 32H, AC = 1, CY = 1
DA    A            ; add 06H: 38H; CY = 1, add 60H
                   ; A = 98H, CY = 1
```

Result: CY=1 A =98H, which is 1 98H, the largest possible sum of two packed BCD bytes.

Summary of the DA A cases

Case	Numbers	Trigger	Adjustment
1	25 + 34	None	None
2	27 + 15	Low nibble > 9	+06H
3	29 + 18	AC = 1	+06H
4	52 + 73	High nibble > 9	+60H
5	91 + 92	CY = 1	+60H
6	78 + 69	AC = 1 and high nibble > 9	+66H
7	99 + 99	AC = 1 and CY = 1	+66H

> **Multi-digit BCD:** to add numbers of more than two digits, add byte by byte starting with the lowest byte. Use ADD then DA A for the lowest byte, and ADDC then DA A for each higher byte, so that the carry passes from one byte to the next.

8. Summary

Item	Summary
ADDC	A = A + source + CY; flags CY, AC, OV, P; CLR C before a normal addition
Multi-byte addition	Add the lowest byte first, then ADDC for each higher byte
Packed BCD	Two decimal digits per byte (59 = 59H)
Unpacked BCD	One decimal digit per byte, upper nibble 0 (59 = 05H, 09H)
DA A	After ADD or ADDC on packed BCD: +06H if low nibble > 9 or AC = 1, then +60H if high nibble > 9 or CY = 1

5. Multiplication: MUL

```
MUL   AB            ; A x B
```

The 16-bit product is split over the two registers: the high byte goes to B and the low byte goes to A.

Register	Holds
B	High byte of the product
A	Low byte of the product

Flags: OV = 1 if the product is greater than FFH (that is, if B is not zero). OV = 0 if the product fits in A. CY is always cleared.

Example 1: 25H x 33H

```
MOV    A, #25H
MOV    B, #33H
MUL    AB            ; A = 5FH, B = 07H    (product = 075FH)
```

Long multiplication in hex:

```
      25H
  x   33H
  ---------
      06FH      (25H x 3)
  +   6F0H       (25H x 30H)
  ---------
     75FH      07H -> B, 5FH -> A
```

> **Added:** in decimal this is 37 x 51 = 1887 = 075FH, which confirms the result.

Example 2: FCH x 2EH

```
MOV    A, #0FCH
MOV    B, #2EH
MUL    AB            ; A = 48H, B = 2DH, OV = 1
```

Example 3: 4 x 5

```
MOV    A, #4
MOV    B, #5
MUL    AB            ; A = 14H, B = 00H    (20 = 0014H, OV = 0)
```

4. Division: DIV

```
DIV   AB            ; A / B
```

Register	Holds
A	Quotient (the result)
B	Remainder

Flags: after DIV, if OV = 1 then B was 0, which means division by zero. This is an error and is not allowed; the results in A and B are undefined. CY is always cleared.

Example: 37 / 10

```
MOV   A, #37
MOV   B, #10
DIV   AB          ; A = 03H (quotient), B = 07H (remainder)
```

Use 1: testing whether a number is even or odd

Divide the number by 2. If the remainder is 0, the number is even; otherwise it is odd.

```
        MOV   A, #__        ; the number to test
        MOV   B, #2
        DIV   AB            ; B = remainder (0 or 1)
        MOV   A, B
        JZ    EVEN          ; remainder = 0 -> even
ODD:    ...                 ; code for an odd number
        SJMP  DONE          ; skip the even block
EVEN:   ...                 ; code for an even number
DONE:   SJMP  $
```

Use 2: converting hex (binary) to unpacked BCD (decimal)

Dividing repeatedly by 10 peels off the decimal digits one at a time, starting with the ones digit. This is shown in the next section.

Example: convert R3 from hex to decimal

Task: convert the contents of R3 from hex to decimal (unpacked BCD) and save the digits in R5, R6 and R7.

Example: R3 = FEH = 254. The digits are 2, 5 and 4, so R5 = 4 (ones), R6 = 5 (tens) and R7 = 2 (hundreds).

```
MOV   A, R3
MOV   B, #10
DIV   AB          ; A = 25 (19H), B = 4
MOV   R5, B       ; R5 = 4  (ones digit)
MOV   B, #10
DIV   AB          ; A = 2, B = 5
MOV   R6, B       ; R6 = 5  (tens digit)
MOV   R7, A       ; R7 = 2  (hundreds digit)
```

Step	A (quotient)	B (remainder)	Saved in
254 / 10	25	4	R5 = 4 (ones)
25 / 10	2	5	R6 = 5 (tens)
Last quotient	2	-	R7 = 2 (hundreds)

The reverse direction: keypad digits to one number

When a user presses 2 and then 5 on a keypad, the two digits arrive as separate unpacked BCD values, for example R2 = 02H (tens) and R3 = 05H (ones). To get the single number 25, multiply the tens digit by 10 and add the ones digit: 10 x 2 = 20, and 20 + 5 = 25.

```
MOV   A, R2        ; tens digit (02H)
MOV   B, #10
MUL   AB           ; A = 20 (14H)
ADD   A, R3        ; A = 20 + 5 = 25 (19H)
```

5. Logical instructions

ANL: AND logic

```
ANL  destination, source
```

The destination is A or a memory location. Each bit of the destination is ANDed with the matching bit of the source.

a	b	a AND b
0	0	0
0	1	0
1	0	0
1	1	1

Example: C6H AND 7AH

```
MOV   A, #0C6H
ANL   A, #7AH      ; A = 42H

    1100 0110      (C6H)
AND 0111 1010      (7AH)
    ---------
    0100 0010      (42H)
```

Use 1: masking (clearing) selected bits

ANDing a bit with **0** clears it. ANDing a bit with **1** leaves it unchanged. So the mask has 0 in every position you want to clear.

```
MOV   A, #32H
ANL   A, #0F0H     ; A = 30H   (low nibble cleared)

    0011 0010      (32H)
AND 1111 0000      (mask)
    ---------
    0011 0000      (30H)
```

Example: clear bits 0, 1 and 7 of A

Put 0 in bit positions 0, 1 and 7 of the mask and 1 everywhere else:

```
ANL   A, #0111 1100B
```

```
ANL   A, #7CH        ; same mask written in hex
```

Use 2: converting ASCII to unpacked BCD (decimal)

The ASCII codes of the digits 0 to 9 are 30H to 39H. Clearing the upper nibble leaves the digit itself.

```
MOV   A, #32H       ; ASCII code of the character "2"
ANL   A, #0FH       ; A = 02H

    0011 0010       (32H)
AND 0000 1111       (0FH)
    ---------
    0000 0010       (02H)
```

ORL: OR logic

```
ORL  destination, source
```

The destination is A or a memory location.

a	b	a OR b
0	0	0
0	1	1
1	0	1
1	1	1

Use 1: setting (making 1) selected bits

ORing a bit with **1** sets it. ORing a bit with **0** leaves it unchanged.

```
ORL   A, #0001 0001B      ; sets bits 4 and 0
```

Use 2: converting unpacked BCD (decimal) to ASCII

Setting bits 5 and 4 (adding 30H) turns a digit 0 to 9 into its ASCII code.

```
MOV   A, #2
ORL   A, #30H       ; A = 32H, the ASCII code of "2"
```

> ANL and ORL together convert between ASCII and unpacked BCD in both directions. This is needed whenever you read digits from a keypad or send digits to an LCD or serial port.